1. Forward Kinematics:
    - To run, Please execute `visualize_6dof_robot(DH_parameters); (run the file FORWARD_WORKING.m)`
    - DH parameters can be specified like:

```
DH_parameters = [
   0, 267, 0, -pi/2;   % Joint1: [theta(rad), d(mm), a(mm), alpha(rad)]
   deg2rad(-79.35), 0, 289.5, 0;      % Joint2
   deg2rad(79.35), 0, 77.5, -pi/2;   % Joint3
   0, 342.5, 0, pi/2;  % Joint4
   0, 0, 76, -pi/2;    % Joint5
   0, 97, 0, 0;        % Joint6
   0, 0,300,0; %eff
];
```

- USE SLIDERS TO GIVE INPUT. You can alternatively specify theta in DH parameters to get OUTPUT.

2. Inverse Kinematics:

- To run, execute the function
- File: inv_last_working.mlx
- Inverse Utilises Fsolve to estimate the nearest solution from current configuration. It also displays the wheatere the solution is reachable or not.
- FOLLOW GUI.

3. EXTRA CREDIT: IK with Jacobian

- See file Jacobian.mlx
- To operate, set q_actual=[1 1 1 1 1 1] as position
- EFF_Vel= [1 1 1 1 1 1]; as desired end effector velocity
- The program returns the corresponding joint velocities

4.Extra Credit: Inverse Dynamics with Jacobian

- See file Jacobian.mlx
- To operate, set q_actual=[1 1 1 1 1 1] as position
- F= [1 1 1 1 1 1]; as desired Force and Moment to be imparted at tip of sword.
- The program returns corresponding joint torques.

CODE:

Forward:

```
function visualize_6dof_robot(DH0)
   % Create a figure and store joint angles in UserData
   fig = figure('Name', '6-DOF Robot Visualization', 'NumberTitle', 'off', ...
```

```
        'Position', [1000, 500,1200, 1000]);
    fig.UserData.jointAngles = zeros(7,1); % Initial joint angles (in radians)

    % Axes for 3D plot
    ax = axes(fig, 'XLim', [-1000, 1000], 'YLim', [-1000, 0], 'ZLim', [0,
1300]);
    view(3);
    grid on;
    hold on;
    xlabel('X'); ylabel('Y'); zlabel('Z');

    % Draw initial robot
    plot_robot(DH0, fig.UserData.jointAngles, ax);

    % Create sliders for each joint
    for i = 1:6
        uicontrol('Style', 'slider', ...
            'Min', -180, 'Max', 180, 'Value',DH0(i,1), ...
            'Position', [10, 50 * (7 - i), 120, 20], ...
            'Callback', @(src, ~) update_robot(src, i, DH0, fig, ax), ...
            'TooltipString', sprintf('Joint %d', i));

        % Add text label for each slider
        uicontrol('Style', 'text', ...
            'Position', [140, 50 * (7 - i), 50, 20], ...
            'String', sprintf('Theta %d', i));
    end
end
function update_robot(slider, jointIdx, DH0, fig, ax)
    % Get the current joint angles from the figure's UserData
    jointAngles = fig.UserData.jointAngles;

    % Update the specific joint angle based on slider value
    jointAngles(jointIdx) = deg2rad(get(slider, 'Value'));

    % Save the updated joint angles back to the figure's UserData
    fig.UserData.jointAngles = jointAngles;

    % Clear the previous plot
    cla(ax);

    % Plot the robot with updated joint angles
    plot_robot(DH0, jointAngles, ax);
end
function plot_robot(DH0, jointAngles, ax)
    % Initialize transformation matrix and origin

    origin = [0; 0; 400];  % Starting point (base)
    origin_T= [1 0 0 origin(1);0 0 -1 origin(2); 0 -1 0 origin(3);0 0 0 1 ];
```

```
    points = origin;
    T = origin_T;
    % Update DH matrix with the current joint angles
    DH = DH0;
    DH(:, 1) = DH(:, 1) +jointAngles; % Update theta values

    % Compute forward kinematics and store points
    for i = 1:size(DH, 1)
        % Calculate transformation matrix for the current joint
        Ti = transformation_matrix(DH(i, 1), DH(i, 2), DH(i, 3), DH(i, 4));

        % Update cumulative transformation matrix
        T = T * Ti;

        % Extract end point position
        endEffectorPos = T(1:3, 4);
        points = [points, endEffectorPos];

        % Draw small frame at each joint
        draw_frame(origin_T(1:3,:),ax)
        draw_frame(T(1:3,:), ax);

        % Label each frame number
        %text(endEffectorPos(1), endEffectorPos(2), endEffectorPos(3),
sprintf('Frame %d', i), 'FontSize', 8);
    end

    % Plot the robot "sticks"
    plot3(ax, points(1,:), points(2,:), points(3,:), '-o', ...
          'LineWidth', 2, 'MarkerSize', 6);
end
function draw_frame(T_frame, ax)
    origin = T_frame(:,4);

    x_axis = T_frame(:,1) * 50; % Scale axis length for visibility
    y_axis = T_frame(:,2) * 50;
    z_axis = T_frame(:,3) * 50;
    quiver3(ax, origin(1), origin(2), origin(3), x_axis(1), x_axis(2),
x_axis(3), 'r');
    quiver3(ax, origin(1), origin(2), origin(3), y_axis(1), y_axis(2),
y_axis(3), 'g');
    quiver3(ax, origin(1), origin(2), origin(3), z_axis(1), z_axis(2),
z_axis(3), 'b');
end
function T = transformation_matrix(theta_deg, d_mm, a_mm, alpha_deg)
    % Convert angles to radians
    theta = (theta_deg);
    alpha = (alpha_deg);
```

```
   % Define the transformation matrix based on DH parameters
   T = [cos(theta), -cos(alpha)*sin(theta), sin(alpha)*sin(theta),
a_mm*cos(theta);
        sin(theta), cos(alpha)*cos(theta), -sin(alpha)*cos(theta),
a_mm*sin(theta);
             0   ,         sin(alpha)   ,         cos(alpha)   ,      d_mm;
             0   ,             0        ,             0        ,       1];
end
% Example usage:
% Define the DH parameters for a robot based on your table:
% Define DH parameters for the 7-DOF robot
DH_parameters = [
   0, 267, 0, -pi/2;   % Joint1: [theta(rad), d(mm), a(mm), alpha(rad)]
   deg2rad(-79.35), 0, 289.5, 0;      % Joint2
   deg2rad(79.35), 0, 77.5, -pi/2;   % Joint3
   0, 342.5, 0, pi/2;  % Joint4
   0, 0, 76, -pi/2;    % Joint5
   0, 97, 0, 0;        % Joint6
   0, 0,300,0; %eff
];
% Display the DH parameters
disp(DH_parameters);
% Call visualization function with these parameters:
visualize_6dof_robot(DH_parameters);
%%
```

Inverse:

```
clear; close all; clc;

q= sym('q', [1 6]);

origin = [0; 0; 400];  % Starting point (base can edit this to modify
origin)

   origin_T= [1 0 0 origin(1);

              0 0 -1 origin(2);

              0 -1 0 origin(3);

                       0 0 0 1 ];
```

```
DH_parameters = [

   q(1), 267, 0, -pi/2;   % Joint1: [theta(rad), d(mm), a(mm),
alpha(rad)]

   q(2)+deg2rad(-79.35), 0, 289.5, 0;      % Joint2
```

```
   q(3)+deg2rad(79.35), 0, 77.5, -pi/2;    % Joint3
   q(4), 342.5, 0, pi/2;   % Joint4
   q(5), 0, 76, -pi/2;     % Joint5
   q(6), 97, 0, 0;         % Joint6
   0, 0,300,0; %eff
];
```

```
TEndEff=Forward_kin(DH_parameters,origin_T);
simplifiedExpr = simplify(TEndEff,"Steps",10);
```

```
Target = [1 0 0 origin(1);
              0 0 -1 origin(2);
              0 -1 0 origin(3);
                       0 0 0 1 ];
init_guess= [1 1 200 1 1 1]; %initial guess
```

```
function [res,total_err] = InverseComputer(s,current,target)
   FinalFunc=s-transpose(target);
   q= sym('q', [1 6]);
   matlabFunction(FinalFunc,"File","FOR_genx.m","Vars",{q});
   % CORE INVERTING FUNCTION> EVERYTHING AROUND IS SAVING SYMBOLIC
REP TO
   % FUNCTION FORMAT, THEN ADJUSTING MATRICES DIMENSIONS, AND
EXECUTION
   res= fsolve(@FOR_genx,current);
   res=fsolve(@FOR_genx,res);
   q= sym('q', [1 6]);
```

```
   error=subs(FinalFunc,q,res(1:6));
   total_err=sum(error, 'all');
   save('errrrrrr.mat', 'total_err');
end
```

```
function O = LinearizeHE(T) %Converts 4x4 Tf into 1x6 Vector
   % Ensure the rotation matrix is valid (3x3)
   R=T(1:3,1:3);
   % Extract angles from the rotation matrix
   oy = atan2(-R(3,1), sqrt(R(1,1)^2 + R(2,1)^2)); % Pitch (oy)
       ox = atan2(R(3,2), R(3,3)); % Roll (ox)
       oz = atan2(R(2,1), R(1,1)); % Yaw (oz)
   O=[T(1:3,4);ox;oy;oz];
end
```

```
s=LinearizeHE(simplifiedExpr);
t=[100 100 0 1 0 1] %LinearizeHE(Target);
FinalFunc=s-transpose(t);
matlabFunction(FinalFunc,"File","FOR_gen.m","Vars",{q}); % Convert
symbol to function
```

```
%ans= fsolve(@FOR_gen,[0 0 0 0 0 0]);
% ans=fsolve(@FOR_gen,ans);
% a2=s-transpose(t);
% error = subs(a2,q,ans)
% ans
```

```
function plot_robot(DH0, jointAngles, ax,origin_T)
   % Initialize transformation matrix and origin


   origin = origin_T(1:3,4);
   points = origin;
   T = origin_T;
   % Update DH matrix with the current joint angles
   DH = DH0;
   DH(:, 1) = DH(:, 1) +jointAngles; % Update theta values


   % Compute forward kinematics and store points
   for i = 1:size(DH, 1)
       % Calculate transformation matrix for the current joint
       Ti = transformation_matrix(DH(i, 1), DH(i, 2), DH(i, 3), DH(i,
4));


       % Update cumulative transformation matrix
       T = T * Ti;


       % Extract end point position
       endEffectorPos = T(1:3, 4);
       points = [points, endEffectorPos];


       % Draw small frame at each joint
       draw_frame(origin_T(1:3,:),ax)
       draw_frame(T(1:3,:), ax);
```

```
        % Label each frame number
        %text(endEffectorPos(1), endEffectorPos(2), endEffectorPos(3),
sprintf('Frame %d', i), 'FontSize', 8);
    end

    % Plot the robot "sticks"
    plot3(ax, points(1,:), points(2,:), points(3,:), '-o', ...
          'LineWidth', 2, 'MarkerSize', 6);
end
```

```
function draw_frame(T_frame, ax)
    origin = T_frame(:,4);

    x_axis = T_frame(:,1) * 50; % Scale axis length for visibility
    y_axis = T_frame(:,2) * 50;
    z_axis = T_frame(:,3) * 50;

    quiver3(ax, origin(1), origin(2), origin(3), x_axis(1), x_axis(2),
x_axis(3), 'r');
    quiver3(ax, origin(1), origin(2), origin(3), y_axis(1), y_axis(2),
y_axis(3), 'g');
    quiver3(ax, origin(1), origin(2), origin(3), z_axis(1), z_axis(2),
z_axis(3), 'b');
end
```

```
function T = transformation_matrix(theta_deg, d_mm, a_mm, alpha_deg)
    % Convert angles to radians
    theta = (theta_deg);
    alpha = (alpha_deg);
```

```
    % Define the transformation matrix based on DH parameters
    T = [cos(theta), -cos(alpha)*sin(theta), sin(alpha)*sin(theta), a_mm*cos(theta);
         sin(theta), cos(alpha)*cos(theta), -sin(alpha)*cos(theta), a_mm*sin(theta);
              0    ,          sin(alpha)   ,          cos(alpha)   , d_mm;
              0    ,              0        ,              0        , 1];
end
```

```
function robot_inverse_visualization(s,DH0,origin_T)
    % Initial DH parameters for the 6-DOF robot [theta, d, a, alpha]
```

```
    % Create a figure for the UI
    fig = figure('Name', '6-DOF Robot INVERSE KINEMATICS Visualization', 'NumberTitle', 'off', ...
                 'Position', [1000, 500,1200, 1000]);
    fig.UserData.jointAngles = zeros(7,1); % Initial joint angles (in radians)
```

```
   % Axes for 3D plot
   ax = axes(fig, 'XLim', [-1500, 1500], 'YLim', [-1500,0], 'ZLim',
[0, 1500]);
   view(3);
   grid on;
   hold on;
   xlabel('X'); ylabel('Y'); zlabel('Z');
```

```
   % Draw initial robot
   plot_robot(DH0, fig.UserData.jointAngles, ax,origin_T);
   uicontrol('Style', 'text', ...
             'String', 'Enter Inverse Coordinates: X Y Z Theta_X
Theta_Y Theta_Z (Note Angles in radians, Y in Negative!)', ...
             'Position', [10, 120, 400, 30], ...
             'FontSize', 12, ...
             'HorizontalAlignment', 'center');
   % Create a text box for user input
   inputBox = uicontrol('Style', 'edit', ...
                        'Position', [10, 80, 400, 30], ...
                        'FontSize', 12, ...
                        'HorizontalAlignment', 'left', ...
                        'TooltipString', 'Enter x y z ax ay az
(space-separated)');
   % Create a text area to display entered values
   Report=uicontrol('Style', 'text', ...
                          'Position', [10, 20, 400, 60], ...
                          'FontSize', 12);
   % Create a button to process the input
   uicontrol('Style', 'pushbutton', ...
```

```
                                'String', 'Submit', ...
                                'Position', [310, 80, 100, 30], ...
                                'FontSize', 12, ...
                                'Callback', @(src,event)
processInput(inputBox, DH0,fig,s,ax,Report,origin_T));



end
```

```
function processInput(inputBox, DH0,fig,s,ax,Report,origin_T)
   % Get the string from the edit box
   inputStr = get(inputBox, 'String');


   % Split the string into individual values (space-separated)
   values = str2num(inputStr); %#ok<ST2NM> Converts string to numeric
array


   if length(values) == 6
       % Display entered values below the text box
       %outputText = findobj(gcf,'Style','text'); % Find the output
text object
       outputText=Report
       try
       J=fig.UserData.jointAngles
       set(outputText,'String',['Entered Values: ',
num2str(values)]);
       % Call inverse_Computer with the entered values
       %set(outputText,'String',['XDXD: ',
num2str(fig.UserData.jointAngles)]);
        [res, error] = InverseComputer(s,transpose(J),values);
```

```
        %res=r(1);
        %error=r(2);
       %fig.UserData.jointAngles=transpose(res);
       %set(outputText,'String',['INVRES: ', num2str(res)]);
       % Visualize the computed DH matrix using visualize_inverse
       cla(ax);
       %plot_robot(DH0, fig.UserData.jointAngles, ax);
       fig.UserData.jointAngles=transpose(res);
       plot_robot(DH0, fig.UserData.jointAngles, ax,origin_T);
       ferr = sprintf('%.3f', error)
       if abs(error) > 0.5
   % Display workspace limit breach message
           set(outputText, 'String', ['Plotted! Total Error: ', ferr,
...
                              '. Workspace limit breach, going to
nearest point.'], ...
                   'ForegroundColor', [1, 0, 0]); % Red text for
warning
       else
   % Display within workspace message
           set(outputText, 'String', ['Plotted! Solution is: ',
num2str(res), ...
                              '. Within workspace. Error:',ferr], ...
                   'ForegroundColor', [0, 0.5, 0]); % Green text for
success
       end
        catch ME
           % Handle any errors during computation or visualization
           set(outputText,'String',['Error: ', ME.message], ...
```

```
                            'ForegroundColor', [1, 0, 0]); % Red for
error
        end


    else
        % Display an error message if not exactly six values are
entered
        outputText = findobj(gcf,'Style','text');
        set(outputText,'String','Error: Please enter exactly six
numeric values.');
    end
end
```

```
DH_parameters = [
    0, 267, 0, -pi/2;    % Joint1: [theta(rad), d(mm), a(mm),
alpha(rad)]
    deg2rad(-79.35), 0, 289.5, 0;       % Joint2
    deg2rad(79.35), 0, 77.5, -pi/2;    % Joint3
    0, 342.5, 0, pi/2;  % Joint4
    0, 0, 76, -pi/2;    % Joint5
    0, 97, 0, 0;        % Joint6
    0, 0,300,0; %eff
];
robot_inverse_visualization(s,DH_parameters,origin_T);
```